PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

Обслуживание базы данных с большим количеством таблиц

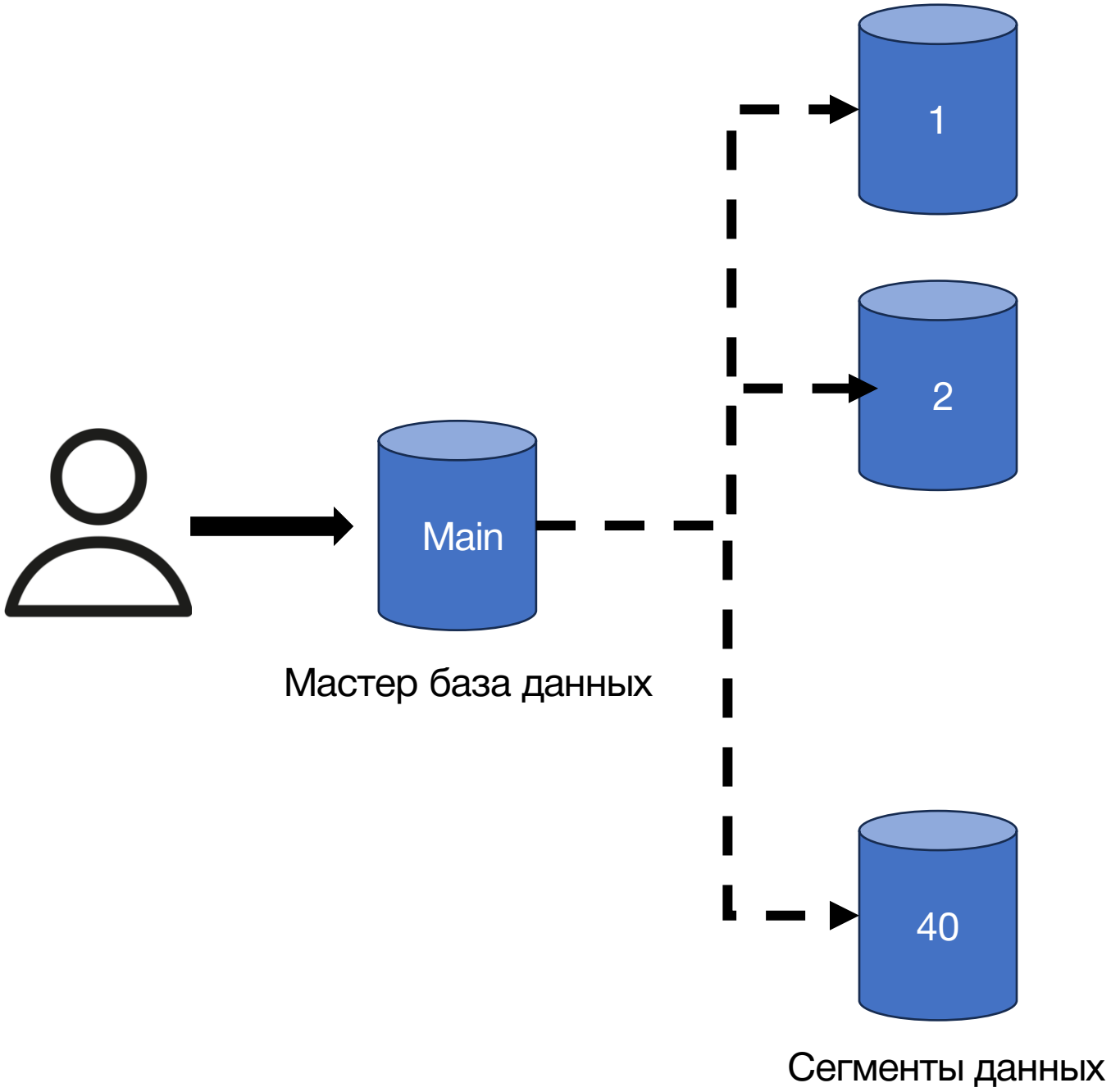
Иван Чувашов

Введение

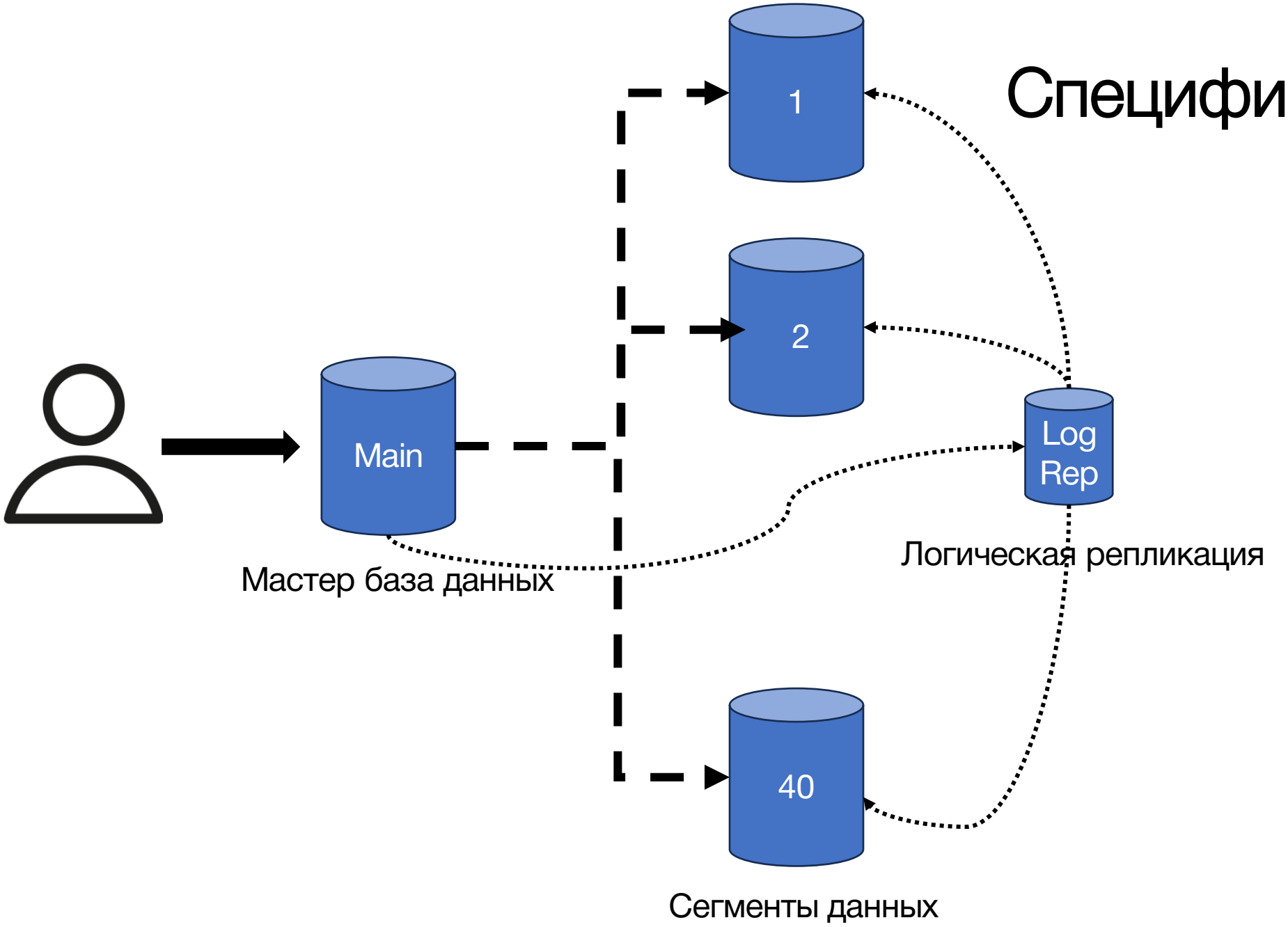
Большое количество таблиц в базе данных требует:

- дополнительного внимания со стороны обслуживания и сопровождения
- корректной работы со статистикой
- изменения параметров системы мониторинга

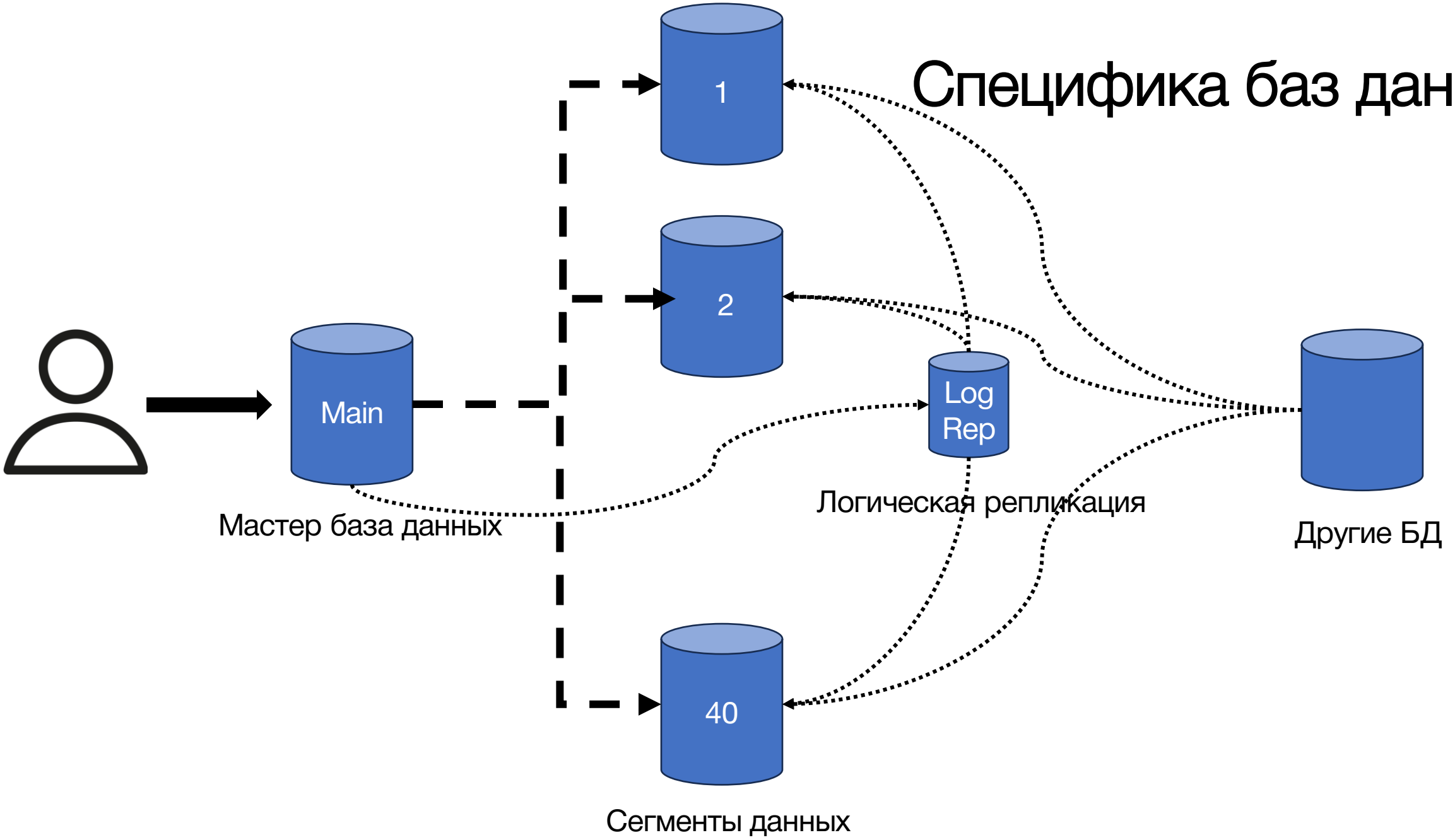
Специфика баз данных



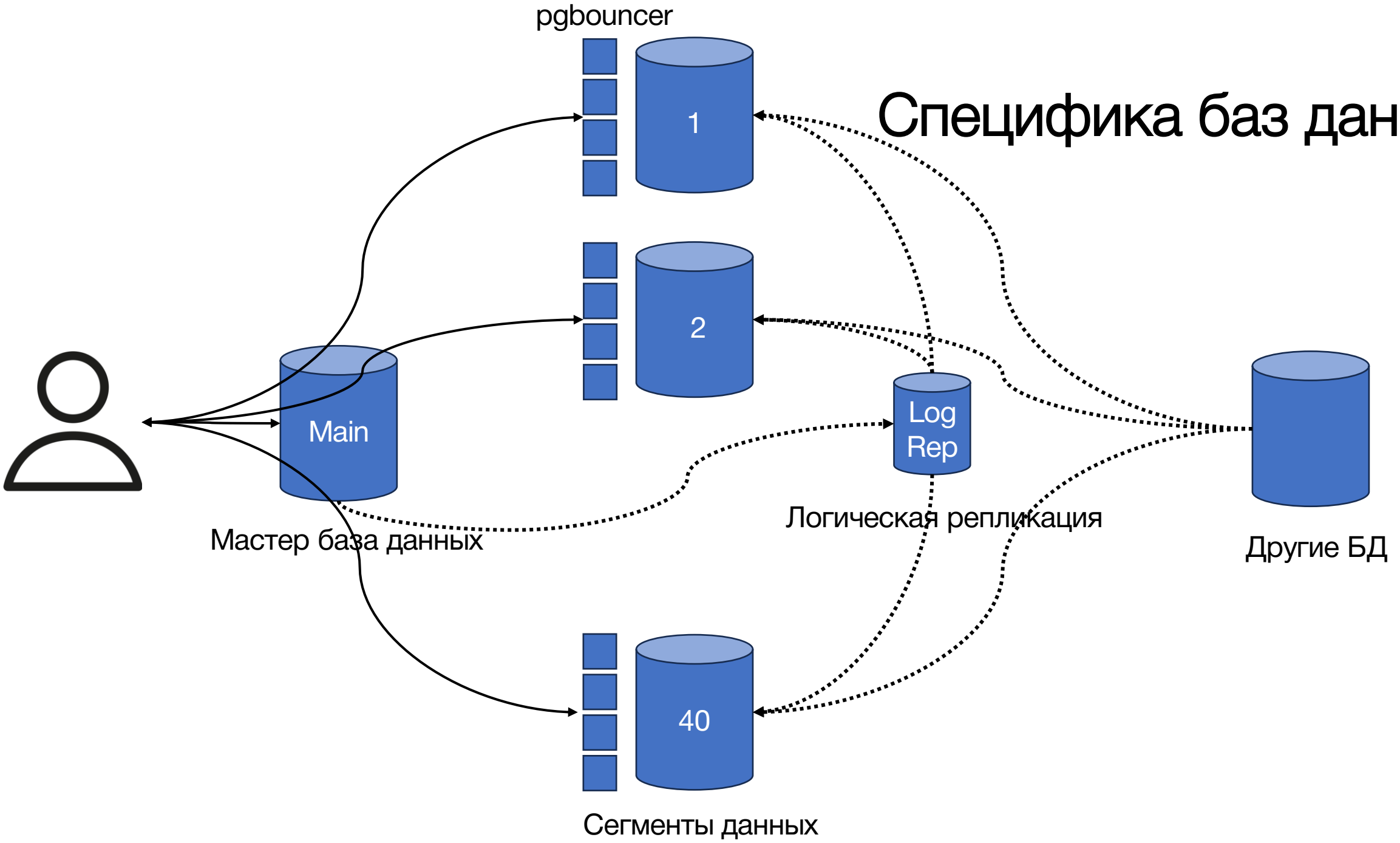
Специфика баз данных



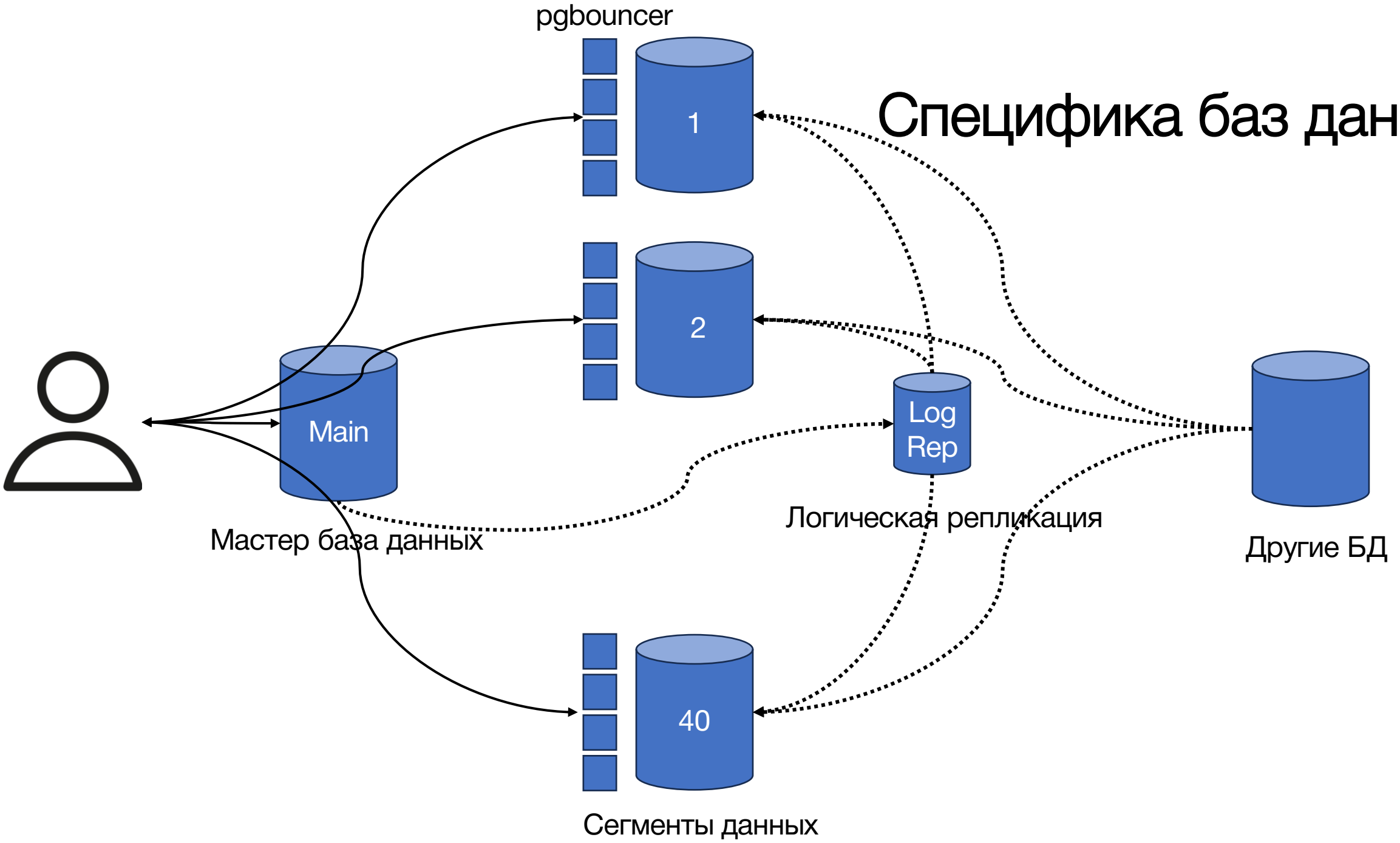
Специфика баз данных

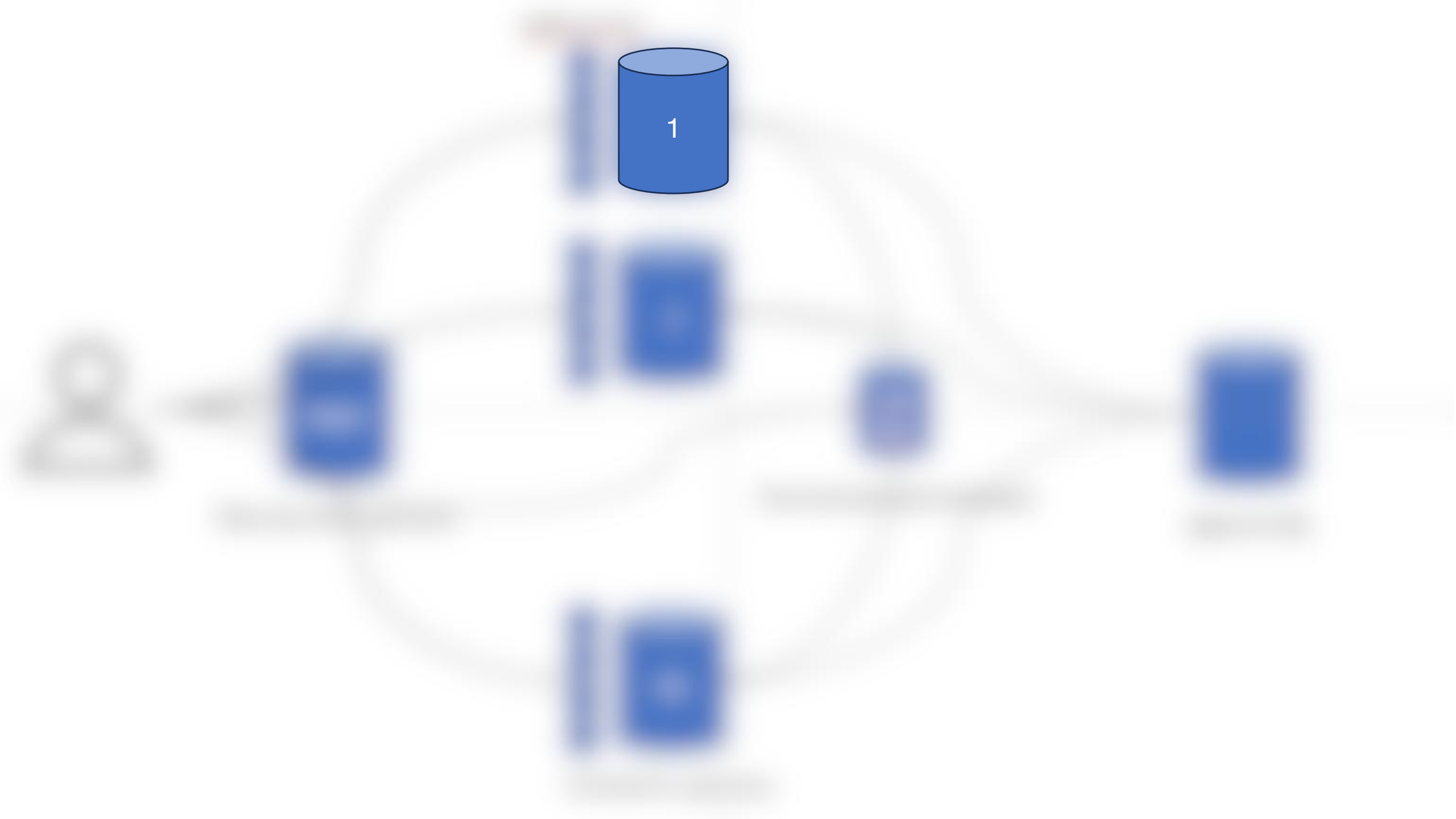


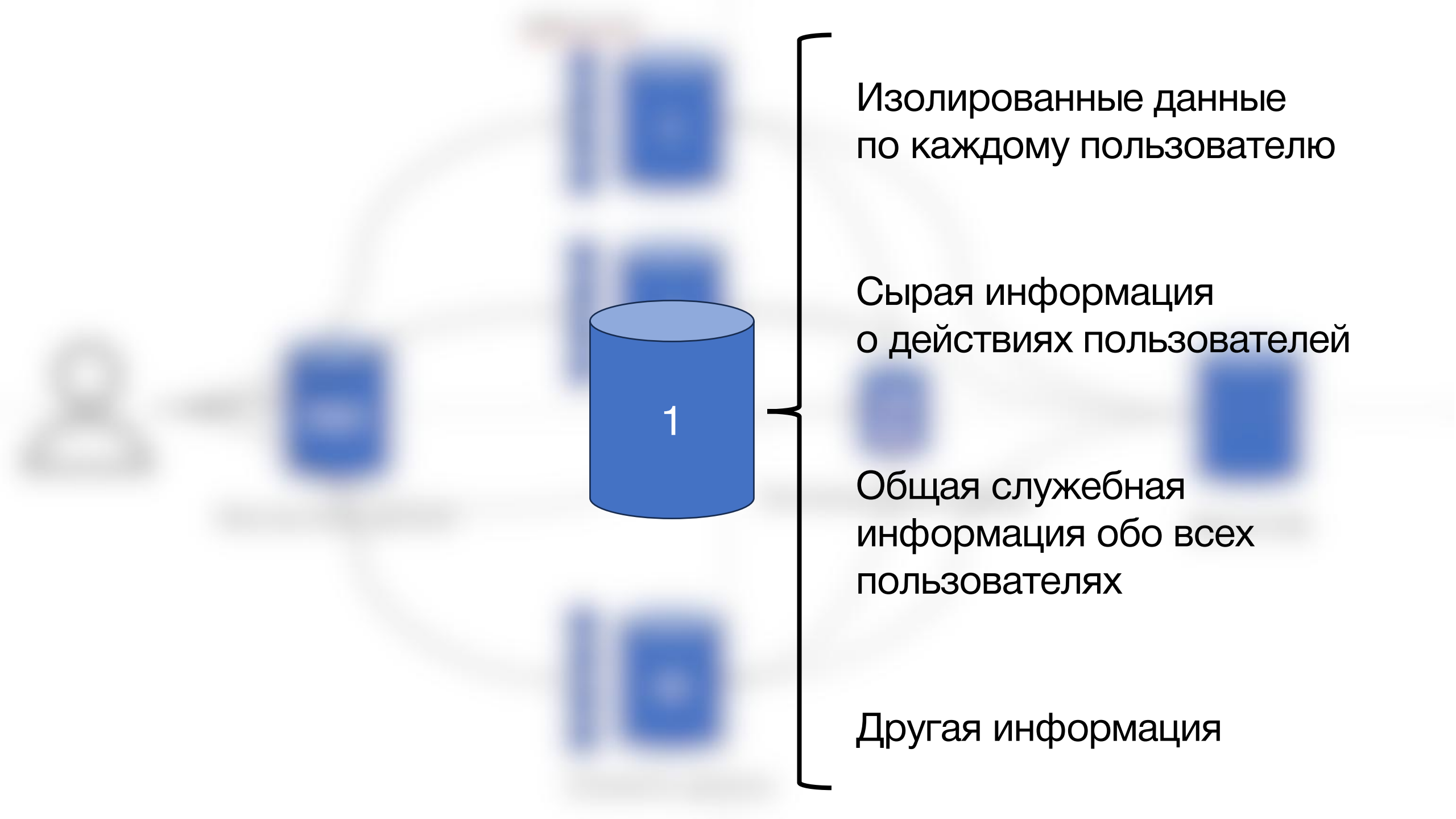
Специфика баз данных

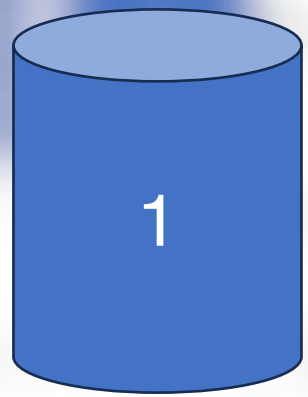


Специфика баз данных




1


1
Изолированные данные
по каждому пользователю
Сырая информация
о действиях пользователей
Общая служебная
информация обо всех
пользователях
Другая информация

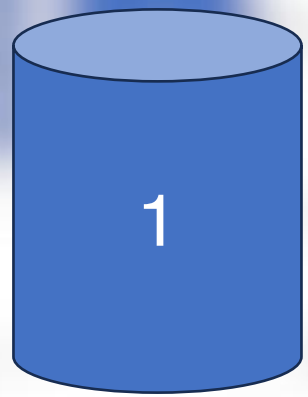


На каждого пользователя в среднем приходится около 50 таблиц.

Каждый пользователь в отдельной схеме

Количество записей от 0 до нескольких миллионов записей

На каждой ноде - около 2000 пользователей

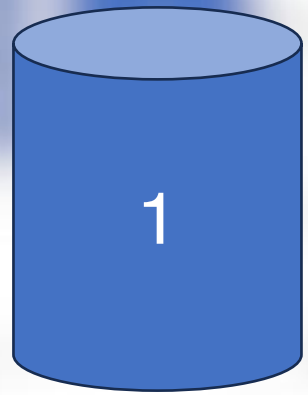


На каждого пользователя в среднем приходится около 50 таблиц

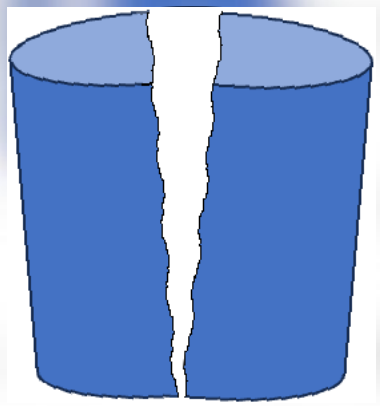
Каждый пользователь в отдельной схеме

Количество записей от 0 до нескольких миллионов записей

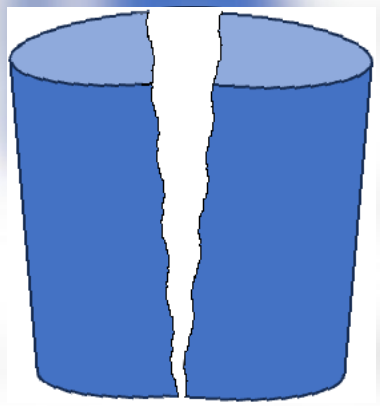
На каждой ноде - около 2000 пользователей



**В каждой базе данных –
около 100 тысяч таблиц**



Если таблиц стало больше, то такая база данных разделяется на две части



Нет смысла разделять на меньшее количество таблиц: сложно поддерживать и обслуживать такое количество БД

Проблемы

Проблемы обслуживания

автовакуум

фриз

Проблемы производительности

нагрузка на CPU/IO

WAL

Проблемы сопровождения

мажорные обновления

раздутие системного каталога

Особенности работы с такими БД

Мажорное обновление баз данных почти невозможно было выполнить

Большое количество таблиц = длительное время выполнения команды pg_upgrade (больше 4 часов)

https://pgconf.ru/talk/1995851#materials

Особенности работы с такими БД

Мажорное обновление баз данных почти невозможно было выполнить

Большое количество таблиц = длительное время выполнения команды pg_upgrade (больше 4 часов)

Изменили pg_upgrade снизили время выполнение до 40 минут

https://pgconf.ru/talk/1995851#materials

Особенности работы с такими БД

Мажорное обновление баз данных почти невозможно было выполнить

Большое количество таблиц = длительное время выполнения команды pg_upgrade (больше 4 часов)

Изменили pg_upgrade снизили время выполнение до 40 минут

PostgreSQL 9.6 –> PostgreSQL 13 –> PostgreSQL 17

https://pgconf.ru/talk/1995851#materials

Мажорное обновление с 9.6 на 13 версию PostgreSQL

- Освободили 20% дискового пространства
- Процессорное время упало на 20%
- Сократили потребление памяти на 10%

Мажорное обновление с PostgreSQL 13 на 17

- Процессорное время упало на 10%
- Ускорение обслуживающих процессов

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов
из-за full_page_writes

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов
из-за full_page_writes

Повышенная
нагрузка на диск

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов
из-за full_page_writes

Повышенная
нагрузка на диск

zfs ?

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества WAL файлов
из-за full_page_writes

Сложности
работы с
хранилищем
WAL файлов

Повышенная
нагрузка на диск

zfs ?

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов из-за full_page_writes

Сложности в оптимизации запросов из-за кардинальности данных в таблицах

PostgreSQL 17

Сложности работы с хранилищем wal файлов

Повышенная нагрузка на диск

zfs ?

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов из-за full_page_writes

Сложности работы с хранилищем wal файлов

Повышенная нагрузка на диск

zfs ?

Сложности в оптимизации запросов из-за кардинальности данных в таблицах

PostgreSQL 17

Выставление индивидуальных настроек для баз данных

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов из-за full_page_writes

Сложности работы с хранилищем wal файлов

Повышенная нагрузка на диск

zfs ?

Быстрое раздутие системного каталога

Выставление индивидуальных настроек для баз данных

Сложности в оптимизации запросов из-за кардинальности данных в таблицах

PostgreSQL 17

Особенности работы с такими БД

Большое количество запросов к большому количеству таблиц

Увеличение количества wal файлов из-за full_page_writes

Сложности работы с хранилищем wal файлов

Повышенная нагрузка на диск

zfs ?

Быстрое раздутие системного каталога

Ночное обслуживание

Сложности в оптимизации запросов из-за кардинальности данных в таблицах

PostgreSQL 17

Выставление индивидуальных настроек для баз данных

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет
значительные ресурсы

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет
значительные ресурсы

Ленивый автовакуум

autovacuum_max_workers = 10
autovacuum_naptime = 10s

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет значительные ресурсы

Пересбор статистики старше одной недели

```sql
select
    quote_ident(n.nspname),
    quote_ident(c.relname),
    c.relnamespace,
    c.oid
from
    pg_class c
join pg_namespace n on
    c.relnamespace = n.oid
left join pg_stat_all_tables st on
    st.relid = c.oid
where
    c.relkind in ('r', 'm')
    and c.reltuples > 1000
    and n.nspname not in ('pg_toast', 'pg_catalog')
    and
(
    (st.last_autoanalyze is null
        and st.last_analyze is null)
    or
        least(
            age(now(), coalesce(st.last_autoanalyze, '2000-01-01'::timestamp with time zone)),
            age(now(), coalesce(st.last_analyze, '2000-01-01'::timestamp with time zone))
        ) > '10 days'
)
order by
    st.last_autoanalyze asc nulls first,
    st.last_autovacuum asc nulls first,
    st.last_analyze asc nulls first,
    st.last_vacuum asc nulls first
```

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет значительные ресурсы

Пересбор статистики старше одной недели

Вакуум таблиц, где давно не было автовакуума

```sql
select
    quote_ident(n.nspname),
    quote_ident(c.relname),
    c.relnamespace,
    c.oid
from
    pg_class c
    join pg_namespace n on c.relnamespace = n.oid
    left join pg_stat_all_tables st on st.relid = c.oid
where
    c.relkind in ('r', 'm', 't')
    and ( (
        c.reltuples > 10000
        and
        (
            (st.last_autoanalyze is null
            and st.last_autovacuum is null
            and st.last_analyze is null
            and st.last_vacuum is null)
        or
            case
            when n.nspname = 'pg_toast'
            or (n.nspname = 'pg_catalog'
                and c.relname <> 'pg_statistic') then
                least(
                    age(now(), coalesce(st.last_autovacuum, '2000-01-01'::timestamp with time zone)),
                    age(now(), coalesce(st.last_vacuum, '2000-01-01'::timestamp with time zone))
                ) > '10 days'
            else
                least(
                    age(now(), coalesce(st.last_autoanalyze, '2000-01-01'::timestamp with time zone)),
                    age(now(), coalesce(st.last_analyze, '2000-01-01'::timestamp with time zone))
                ) > '10 days'
                or
                least(
                    age(now(), coalesce(st.last_autovacuum, '2000-01-01'::timestamp with time zone)),
                    age(now(), coalesce(st.last_vacuum, '2000-01-01'::timestamp with time zone))
                ) > '10 days'
            end
        )
    ) or (c.reltuples > 1000 and n.nspname in ('pg_toast', 'pg_catalog')
     and ((st.last_autoanalyze is null and st.last_autovacuum is null and st.last_analyze is null and st.last_vacuum is null) ) ) )
order by st.last_autoanalyze asc nulls first, st.last_autovacuum asc nulls first, st.last_analyze asc nulls first, st.last_vacuum asc nulls first
```

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет значительные ресурсы

Пересбор статистики старше одной недели

Вакуум таблиц, где давно не было автовакуума

Запуск pg_repack для «раздутых» таблиц и индексов

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет значительные ресурсы

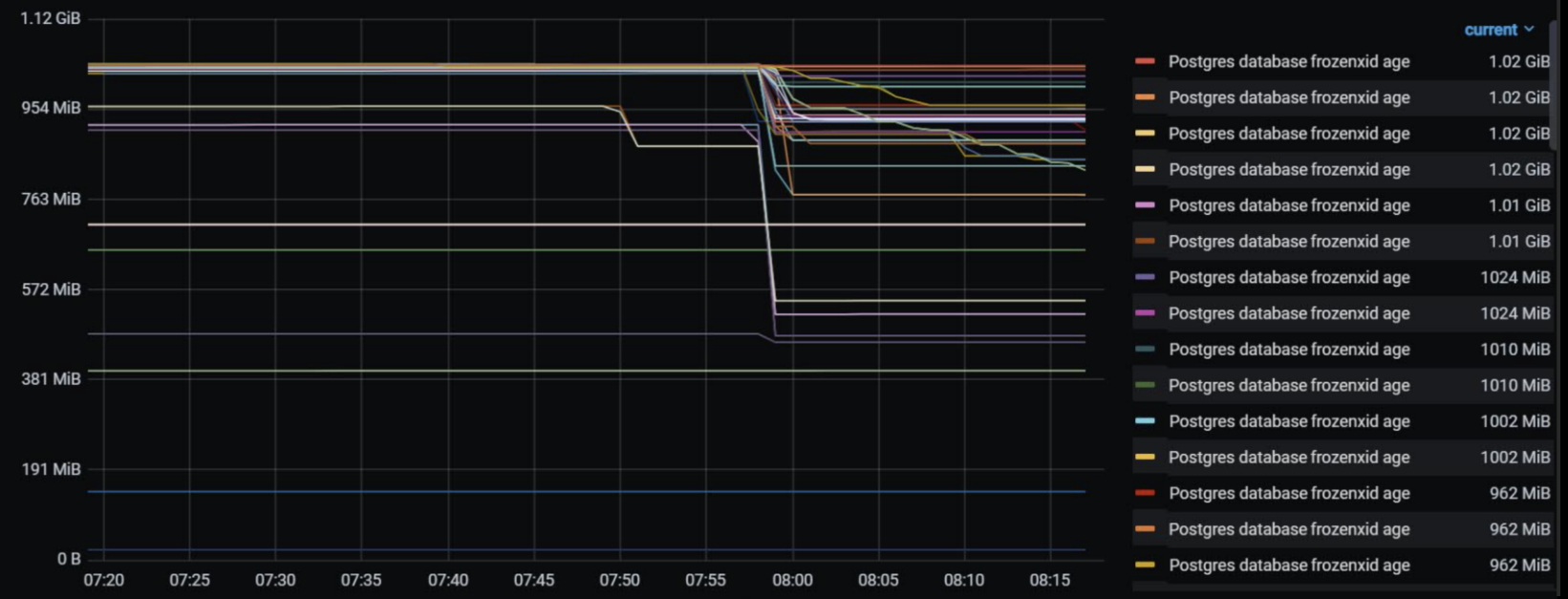
Пересбор статистики старше одной недели

Вакуум таблиц, где давно не было автовакуума

Запуск pg_repack для «раздутых» таблиц и индексов

Freeze таблиц

Postgres - frozenxid age



Postgres - frozenxid age

Особенности работы с такими БД

Работа обслуживающих процессов

- Автовакуум потребляет значительные ресурсы
  - Пересбор статистики старше одной недели
  - Вакуум таблиц, где давно не было автовакуума
  - Запуск pg_repack для «раздутых» таблиц и индексов
  - Freeze таблиц
- Сбор статистики pg_stat_tmp

Особенности работы с такими БД

Работа обслуживающих процессов

Автовакуум потребляет значительные ресурсы

Сбор статистики pg_stat_tmp

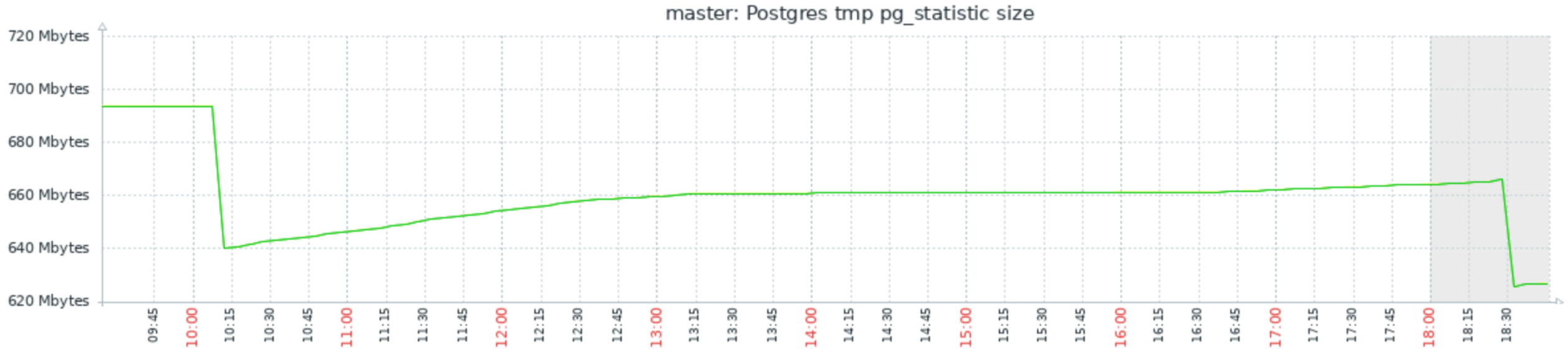
> PostgreSQL 14

Пересбор статистики старше одной недели

Вакуум таблиц, где давно не было автовакуума

Запуск pg_repack для «раздутых» таблиц и индексов

Freeze таблиц



		last	min	avg	max
Postgres tmp pg_statistic size	[all]	626.65 Mbytes	625.75 Mbytes	659.94 Mbytes	693.71 Mbytes

Особенности работы с такими БД

Мониторинг

Долгие запросы к системному каталогу

Особенности работы с такими БД

Мониторинг

Долгие запросы к системному каталогу

Блокировка системного каталога

Особенности работы с такими БД

Мониторинг

Долгие запросы к системному каталогу

Блокировка системного каталога

Собственная разработка с индивидуальным подходом для баз данных с большим количеством таблиц

Особенности работы с такими БД

Мониторинг

Долгие запросы к системному каталогу

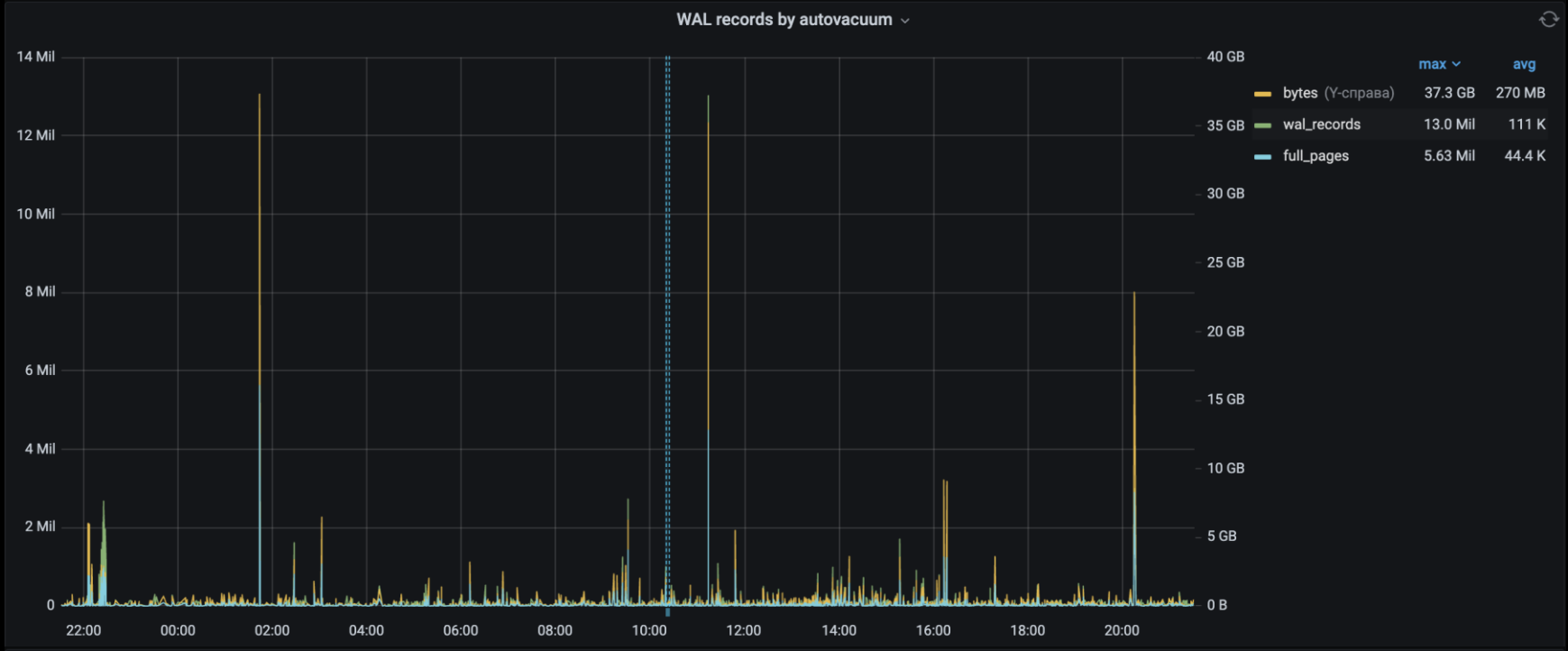
Блокировка системного каталога

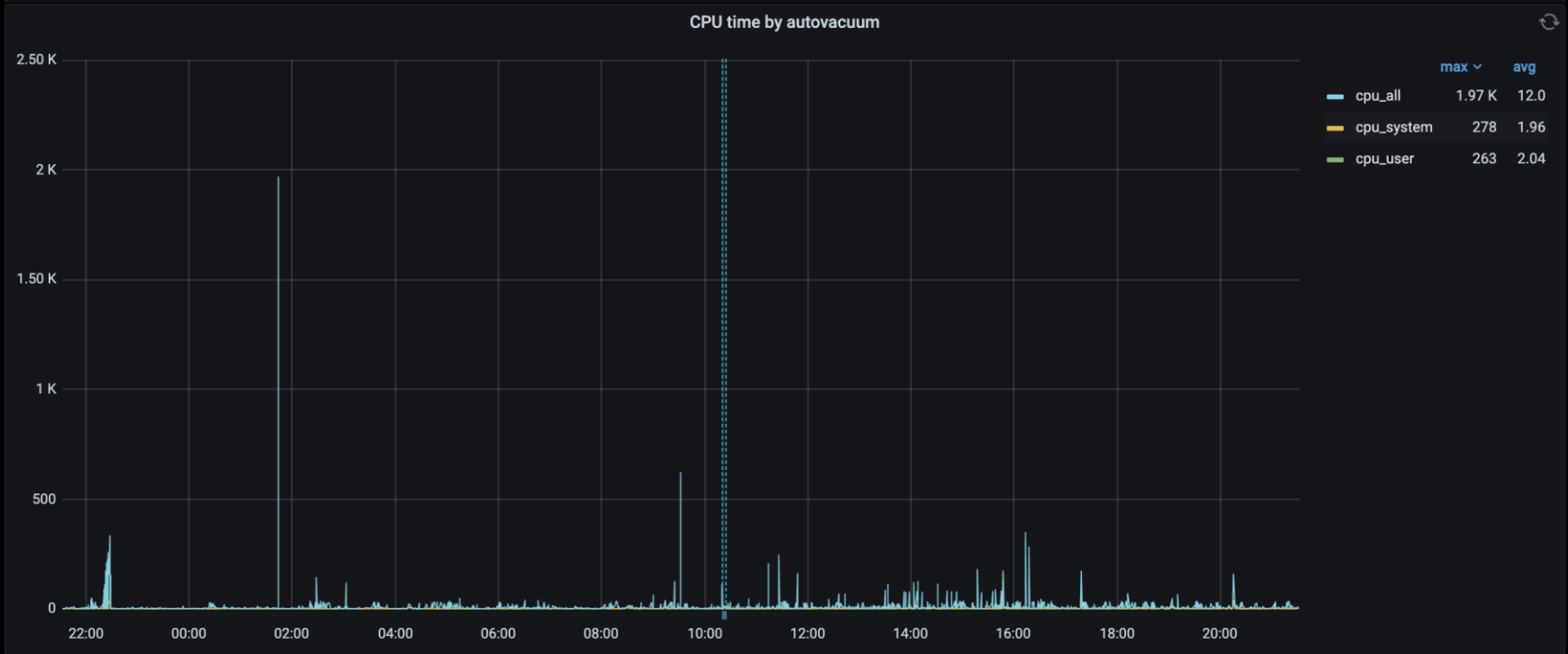
Собственная разработка с индивидуальным подходом для баз данных с большим количеством таблиц

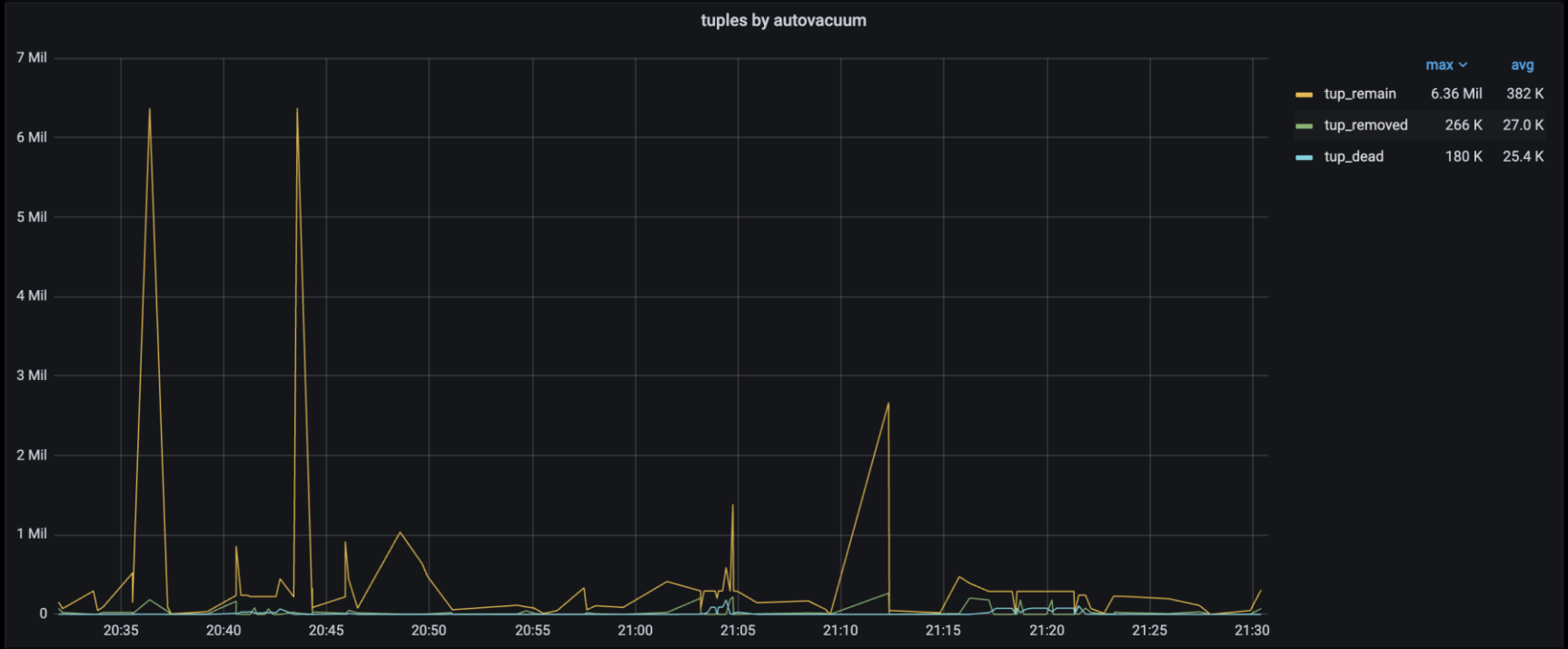
хранение промежуточных результатов в таблицах

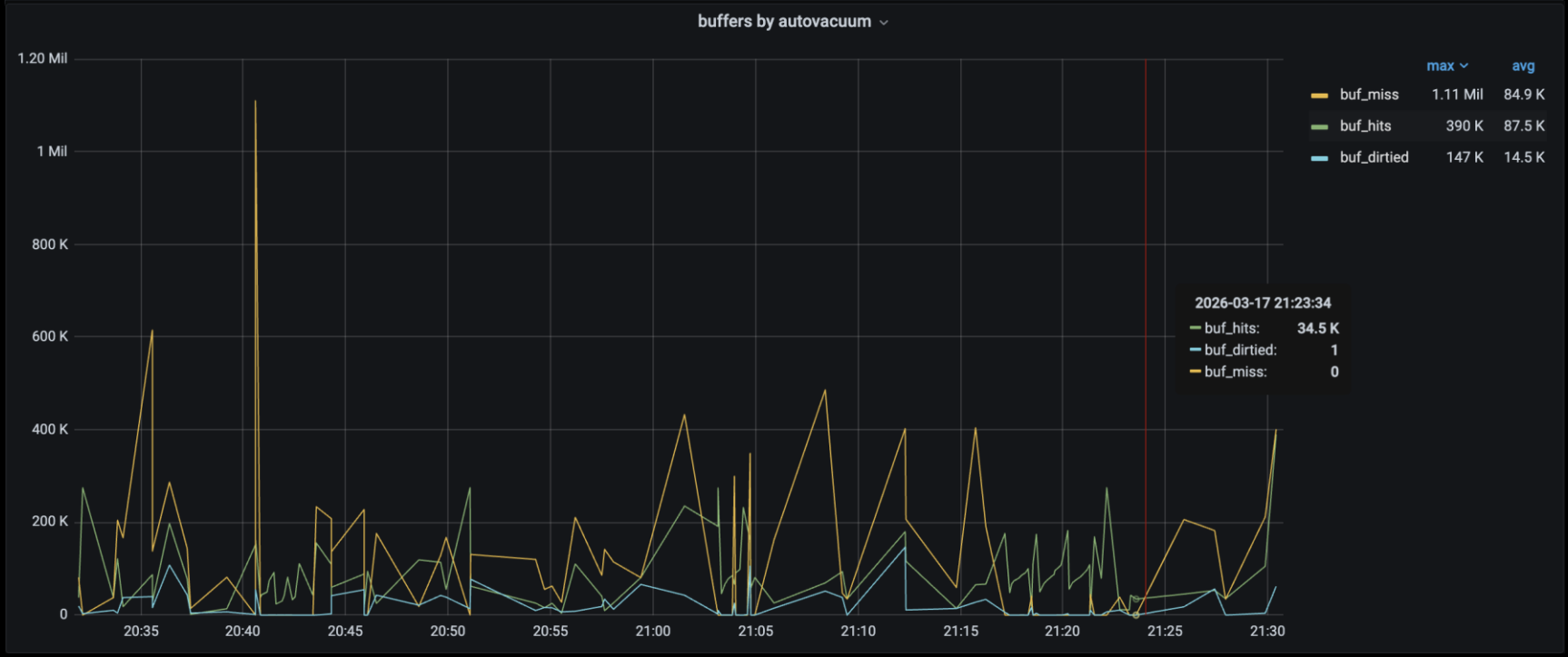
Лог работы автовакуума

2026-02-18 10:59:18.798 MSK,,,41125,,69bc5a23.44841,1,,2026-02-18 10:59:14
MSK,512/657981,0,LOG,00000,"automatic vacuum of table "postgres.test"": index scans: 1
pages: 0 removed, 186606 remain, 75975 scanned (40.71% of total)
tuples: 205383 removed, 6423644 remain, 0 are dead but not yet removable
removable cutoff: 23885002, which was 184 XIDs old when operation ended
frozen: 9452 pages from table (5.07% of total) had 189359 tuples frozen
index scan needed: 18863 pages from table (10.11% of total) had 220511 dead item identifiers removed
index ""ix_test_id"": pages: 43899 in total, 92 newly deleted, 4672 currently deleted, 4664 reusable
index ""pk_test_batch"": pages: 29654 in total, 849 newly deleted, 5137 currently deleted, 5137 reusable
I/O timings: read: 2036.379 ms, write: 39.558 ms
avg read rate: 196.713 MB/s, avg write rate: 62.323 MB/s
buffer usage: 155427 hits, 104781 misses, 33197 dirtied
WAL usage: 81777 records, 33300 full page images, 189806172 bytes
system usage: CPU: user: 1.33 s, system: 1.57 s, elapsed: 4.16 s",,,,,,,,,"","autovacuum worker",,0


WAL records by autovacuum
14 Mil
12 Mil
10 Mil
8 Mil
6 Mil
4 Mil
2 Mil
0
40 GB
35 GB
30 GB
25 GB
20 GB
15 GB
10 GB
5 GB
0 B
22:00
00:00
02:00
04:00
06:00
08:00
10:00
12:00
14:00
16:00
18:00
20:00
max
avg
bytes (Y-справа) 37.3 GB 270 MB
wal_records 13.0 Mil 111 K
full_pages 5.63 Mil 44.4 K


CPU time by autovacuum
2.50 K
2 K
1.50 K
1 K
500
0
22:00
00:00
02:00
04:00
06:00
08:00
10:00
12:00
14:00
16:00
18:00
20:00
max
avg
cpu_all 1.97 K 12.0
cpu_system 278 1.96
cpu_user 263 2.04


tuples by autovacuum
7 Mil
6 Mil
5 Mil
4 Mil
3 Mil
2 Mil
1 Mil
0
20:35
20:40
20:45
20:50
20:55
21:00
21:05
21:10
21:15
21:20
21:25
21:30
max
avg
tup_remain 6.36 Mil 382 K
tup_removed 266 K 27.0 K
tup_dead 180 K 25.4 K


buffers by autovacuum
1.20 Mil
1 Mil
800 K
600 K
400 K
200 K
0
20:35
20:40
20:45
20:50
20:55
21:00
21:05
21:10
21:15
21:20
21:25
21:30
max
avg
buf_miss 1.11 Mil 84.9 K
buf_hits 390 K 87.5 K
buf_dirtied 147 K 14.5 K
2026-03-17 21:23:34
buf_hits: 34.5 K
buf_dirtied: 1
buf_miss: 0

Статистика для 500 таблиц

schema_name	table_name	table_size	total_size_with_indexes	size_bytes	approx_rows
pg_catalog	pg_attribute	3432 kB	10 MB	3514368	24126
pg_catalog	pg_proc	792 kB	1232 kB	811008	3330
pg_catalog	pg_class	728 kB	2640 kB	745472	3915
pg_catalog	pg_depend	544 kB	4488 kB	557056	9209
pg_catalog	pg_index	384 kB	808 kB	393216	2164
pg_catalog	pg_description	360 kB	608 kB	368640	5205
pg_catalog	pg_type	304 kB	784 kB	311296	1617
pg_catalog	pg_attrdef	272 kB	608 kB	278528	1000
pg_catalog	pg_statistic	208 kB	360 kB	212992	422
pg_catalog	pg_collation	120 kB	256 kB	122880	841
pg_catalog	pg_rewrite	112 kB	728 kB	114688	145

```
test=# explain (buffers,analyze)select * from schema_0008.table_25 where data='c81';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on table_25  (cost=4.48..89.79 rows=24 width=60) (actual time=0.035..0.077 rows=23 loops=1)
   Recheck Cond: (data = 'c81'::text)
   Heap Blocks: exact=23
   Buffers: shared hit=25
   ->  Bitmap Index Scan on table_25_data_idx  (cost=0.00..4.47 rows=24 width=0) (actual time=0.022..0.022 rows=23 loops=1)
         Index Cond: (data = 'c81'::text)
         Buffers: shared hit=2
 Planning:
   Buffers: shared hit=81
 Planning Time: 0.436 ms
 Execution Time: 0.106 ms
(11 rows)
```

Статистика для 5000 таблиц

schema_name	table_name	table_size	total_size_with_indexes	size_bytes	approx_rows
pg_catalog	pg_attribute	29 MB	40 MB	30695424	213126
pg_catalog	pg_class	6392 kB	10168 kB	6545408	35415
pg_catalog	pg_depend	4520 kB	11 MB	4628480	76709
pg_catalog	pg_index	3512 kB	4432 kB	3596288	20164
pg_catalog	pg_attrdef	2672 kB	3240 kB	2736128	10000
pg_catalog	pg_type	1976 kB	2832 kB	2023424	10617
pg_catalog	pg_constraint	840 kB	1760 kB	860160	5112
pg_catalog	pg_proc	792 kB	1232 kB	811008	3330
pg_catalog	pg_sequence	440 kB	624 kB	450560	5000
pg_catalog	pg_description	360 kB	608 kB	368640	5205
pg_catalog	pg_statistic	224 kB	384 kB	229376	435

```
test=# explain (buffers,analyze)select * from schema_0008.table_25 where data='c81';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on table_25  (cost=4.48..89.79 rows=24 width=60) (actual time=0.044..0.092 rows=23 loops=1)
   Recheck Cond: (data = 'c81'::text)
   Heap Blocks: exact=23
   Buffers: shared hit=25
   ->  Bitmap Index Scan on table_25_data_idx  (cost=0.00..4.47 rows=24 width=0) (actual time=0.034..0.034 rows=23 loops=1)
         Index Cond: (data = 'c81'::text)
         Buffers: shared hit=2
 Planning:
   Buffers: shared hit=115 dirtied=1
 Planning Time: 0.409 ms
 Execution Time: 0.130 ms
(11 rows)
```

Статистика для 50000 таблиц

schema_name	table_name	table_size	total_size_with_indexes	size_bytes	approx_rows
pg_catalog	pg_attribute	284 MB	392 MB	297648128	1998476
pg_catalog	pg_class	61 MB	100 MB	63913984	350415
pg_catalog	pg_depend	43 MB	112 MB	45285376	715495
pg_catalog	pg_index	34 MB	42 MB	35651584	200164
pg_catalog	pg_attrdef	26 MB	30 MB	27312128	100000
pg_catalog	pg_type	18 MB	25 MB	19169280	100617
pg_catalog	pg_constraint	8184 kB	15 MB	8380416	47675
pg_catalog	pg_sequence	4368 kB	5552 kB	4472832	50000
pg_catalog	pg_proc	792 kB	1232 kB	811008	3330
pg_catalog	pg_description	360 kB	608 kB	368640	5205
pg_catalog	pg_statistic	240 kB	408 kB	245760	435

```
test=# explain (buffers,analyze)select data from schema_0008.table_25 where data='c81';
                                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Index Only Scan using table_25_data_idx on table_25  (cost=0.29..4.71 rows=24 width=4) (actual time=0.111..0.116 rows=23 loops=1)
   Index Cond: (data = 'c81'::text)
   Heap Fetches: 0
   Buffers: shared read=3
 Planning:
   Buffers: shared hit=117 read=8
 Planning Time: 0.986 ms
 Execution Time: 0.180 ms
(8 rows)
```

Статистика для 100000 таблиц

schema_name	table_name	table_size	total_size_with_indexes	size_bytes	approx_rows
pg_catalog	pg_attribute	567 MB	782 MB	594960384	4026625
pg_catalog	pg_class	122 MB	201 MB	127729664	671988
pg_catalog	pg_depend	86 MB	223 MB	90456064	1478319
pg_catalog	pg_index	68 MB	84 MB	71270400	384051
pg_catalog	pg_attrdef	52 MB	61 MB	54616064	196830
pg_catalog	pg_type	36 MB	50 MB	38223872	191252
pg_catalog	pg_constraint	16 MB	30 MB	16744448	98540
pg_catalog	pg_sequence	8728 kB	11 MB	8937472	98477
pg_catalog	pg_proc	792 kB	1232 kB	811008	3330
pg_catalog	pg_description	360 kB	608 kB	368640	5205
pg_catalog	pg_statistic	240 kB	408 kB	245760	435

```
test=# explain (buffers,analyze)select * from schema_0008.table_25 where data='c81';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on table_25  (cost=4.48..89.79 rows=24 width=60) (actual time=0.106..0.368 rows=23 loops=1)
   Recheck Cond: (data = 'c81'::text)
   Heap Blocks: exact=23
   Buffers: shared read=25
   ->  Bitmap Index Scan on table_25_data_idx  (cost=0.00..4.47 rows=24 width=0) (actual time=0.075..0.076 rows=23 loops=1)
         Index Cond: (data = 'c81'::text)
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=111 read=11 dirtied=1
 Planning Time: 0.720 ms
 Execution Time: 0.465 ms
(11 rows)
```

Выводы

- **Актуальность статистики решение проблем производительности:** Пересбор статистики (даже "старой") критически важен для планировщика в условиях перекоса данных
- **Масштабирование:** > 100k таблиц для PostgreSQL - это нормально, если выстроена правильная система обслуживания и мониторинга
- **Debug через логи:** Мониторинг логов автовакуума позволяет предотвращать проблемы до того, как они станут критическими.

Контакты

Tg: @chuvashov_ivan